

NetSage AI — Applied AI + Network Troubleshooting

Industry Problem Statement – Solution Summary

1. Introduction

NetSage AI is an AI-assisted troubleshooting helper for Cisco-style networking lab scenarios. It reads a reported symptom together with topology notes and show-command evidence, identifies a likely fault, indicates the relevant OSI layer, recommends the next diagnostic command, and provides evidence-backed fix steps. Human review remains responsible for approving, editing, or rejecting every diagnosis.

2. Problem Statement

Junior network engineers may know individual networking commands while still struggling to connect an observed symptom to the underlying root cause. The solution organizes troubleshooting evidence into structured cases and presents a repeatable diagnosis-and-review workflow across VLAN, gateway, DHCP, DNS, routing, ACL, NAT, and wireless issues.

3. Objectives

- Create a structured troubleshooting case dataset covering multiple Cisco-style network fault types.
- Use symptoms, topology notes and show-command outputs as evidence.
- Produce root cause, confidence, evidence, OSI layer, next command and fix steps.
- Provide deterministic rule checks for common configuration mistakes.
- Require human review with Accepted, Edited and Rejected outcomes.
- Maintain a responsible-AI correction history.
- Provide a dashboard summarizing issue types, severity and review outcomes.
- Demonstrate a complete troubleshooting flow from case through diagnosis, review, fix and verification.

4. Scope

The solution focuses on Cisco-style networking troubleshooting in a lab-oriented context. The current application contains 35 structured cases spanning VLAN, Gateway, DHCP, DNS, Routing, ACL, NAT and Wireless categories. The core diagnosis path is deterministic and works without an external AI service.

5. Step-by-Step Methodology

Step 1 — Case collection: Build structured cases containing symptoms, topology, command evidence, expected fault, OSI layer, concept/category and severity.

Step 2 — Structured diagnosis: Organize diagnostic output around root cause, confidence, evidence, next command and fix steps.

Step 3 — Deterministic checking: Apply reproducible rules to common VLAN, trunk, gateway, interface, routing, DHCP, ACL, NAT and wireless signals.

Step 4 — Diagnosis and comparison: Generate the available diagnosis from case evidence and compare it with the reference answer.

Step 5 — Human review: Record Accepted, Edited or Rejected and retain correction information for edited/rejected outcomes.

Step 6 — Dashboard and demonstration: Present review progress and issue distribution, then demonstrate a complete case workflow.

6. System Architecture

The application is implemented as a mobile-friendly web application. The case layer stores structured troubleshooting cases and reference answers. A deterministic rule layer evaluates case signals and produces a diagnosis. The review layer captures the human decision and corrections. Dashboard and corrections views summarize the stored review results.

7. Deterministic Rule Engine

The rule engine is designed to be reproducible and explainable. Each rule is a signal check with a defined OSI layer, root-cause statement, next command, fix steps and explanation. Confidence is an evidence-weight calculation rather than a claim of model accuracy.

8. Human Oversight and Responsible AI

Human review is a mandatory control. The reviewer records Accepted, Edited or Rejected, and corrections are retained for cases requiring intervention.

9. Final Review Snapshot

Metric	Final state
Cases reviewed	35 / 35
Accepted	30
Edited	4
Rejected	1
AI / human agreement	86% (application-reported)
Rule coverage	34 / 35
Human-corrected cases	5 (4 Edited + 1 Rejected)

10. User Workflow

1. Open the case list.
2. Select a case.
3. Inspect symptoms, topology and show-command evidence.
4. Run the diagnosis.
5. Review root cause, confidence, OSI layer, evidence, next command and fix steps.
6. Choose Accepted, Edited or Rejected.
7. For Edited, record corrected fields and correction targets.
8. Record the review.
9. Continue through the cases.
10. Use Dashboard and Corrections to inspect outcomes.

11. Technologies and Tools

The project is implemented as a TypeScript/React-based web application with Tailwind/shadcn-style UI components. The submission package includes structured case data, diagnosis prompt documentation, deterministic Python checker documentation and implementation, rule documentation, source exports, and responsible-AI review documentation.

12. Expected Outcomes

The expected outcome is a repeatable troubleshooting assistant that connects observable network evidence to likely faults and actionable next steps while preserving human control and maintaining a transparent record of accepted, corrected and rejected diagnoses.

13. Conclusion

NetSage AI implements the central workflow of the supplied Applied AI + Network Troubleshooting problem: evidence-driven troubleshooting, deterministic checks, structured diagnosis, mandatory human review, correction logging and dashboard reporting. The application is prepared as the main solution artifact, while this summary documents how the solution maps to the requested workflow and evaluation criteria.

Requirement-to-Solution Mapping

Problem-statement requirement	Current solution / status
Case dataset	35 structured cases across the required networking fault categories.
Evidence per case	Symptoms, topology notes, show-command outputs and reference fault information.
Structured diagnosis	Root cause, confidence, evidence, OSI layer, next command and fix steps.
Rule checker	Deterministic signal-based rules plus a standalone Python checker.
Human oversight	Accepted / Edited / Rejected review workflow with persisted reviewer decisions; 35/35 cases reviewed.
Responsible AI	Five recorded human-correction cases: 4 Edited and 1 Rejected. Corrections are retained in the responsible-AI review log.
Dashboard	Issue-type, severity, review and rule-coverage summaries. Final snapshot: 35 reviewed, 86% AI/human agreement, 34/35 rule coverage.
Demo workflow	Case → diagnosis → human review → record → corrections/dashboard.